SENG 201
Software Engineering Foundations

[Week 2, Wednesday]



Yesterday…



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Syntax of class declarations

```
[modifiers*] class Identifier [extends Parent] [implements Interfaces*]
{
   [members*]
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Operators and Strings



→ Arithmetic operators

→ Boolean operators

→ Strings and string operations

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Java (common) operators

Arithmetic

Operator	Meaning
`x++  x--`	increment/decrement, return original value
`++x  --x`	increment/decrement, return new value
`*  /  %`	multiply, divide, modulo
`+  -`	add, substract

Assignment

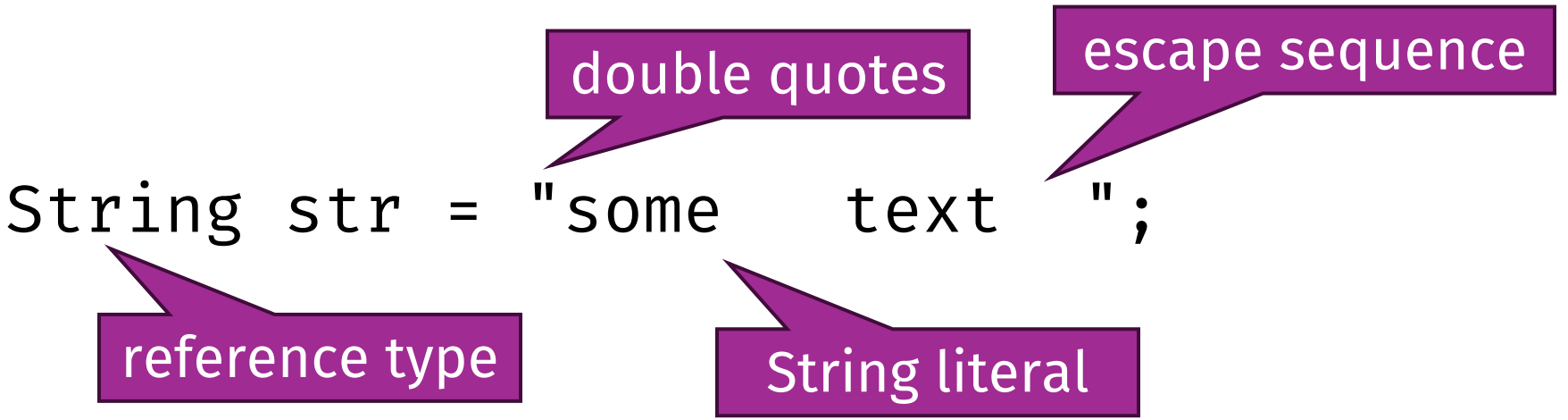
Operator	Meaning
`x = y`	assign y to x
`+=  -=  *=  /=  %=  &=  |=` (`x op= y`)	compute x op y, then assign to x

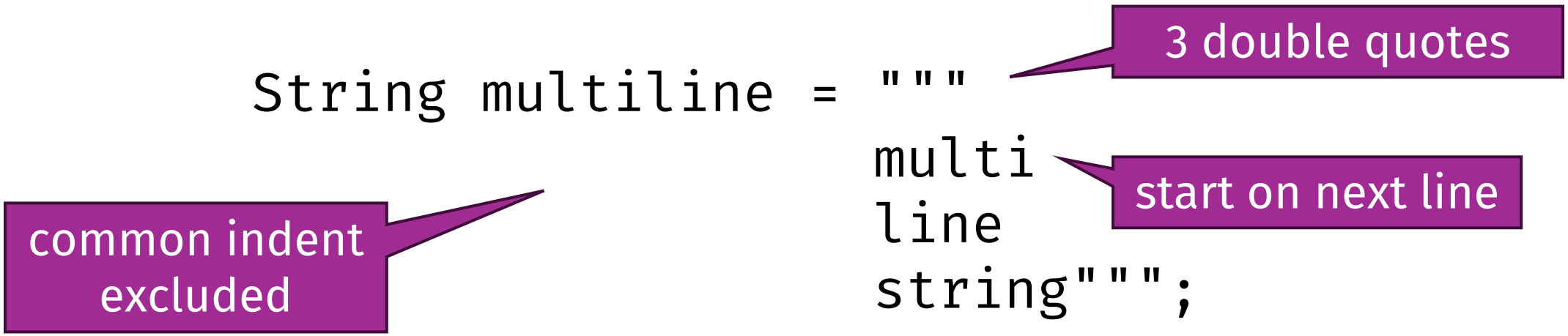
Logical

Operator	Meaning
`!`	NOT
`<  >  <=  >=`	numeric comparison
`&&`	AND (short-circuiting)
`||`	OR (short-circuiting)
`==  !=`	identity equal/unequal

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Strings





© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

String operations

Strings are immutable. All operations return a new String object.

- Concatenation

  ```
  "seng" + "201"
  ```

- Split

  by index: `"my string".substring(start, end)` (end is optional)

  by delimiter: `"path/to/file".split("/")` (regex)

  single character: `"abc".charAt(0)`

- Change case

  ```
  "lowercase".toUpperCase()
  "UPPER".toLowerCase()
  ```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

String operations

- Object to String

  Arrays: `Arrays.toString(someArray)`

  Objects: `myObject.toString()` `// or "" + myObject`

- Primitive types to String

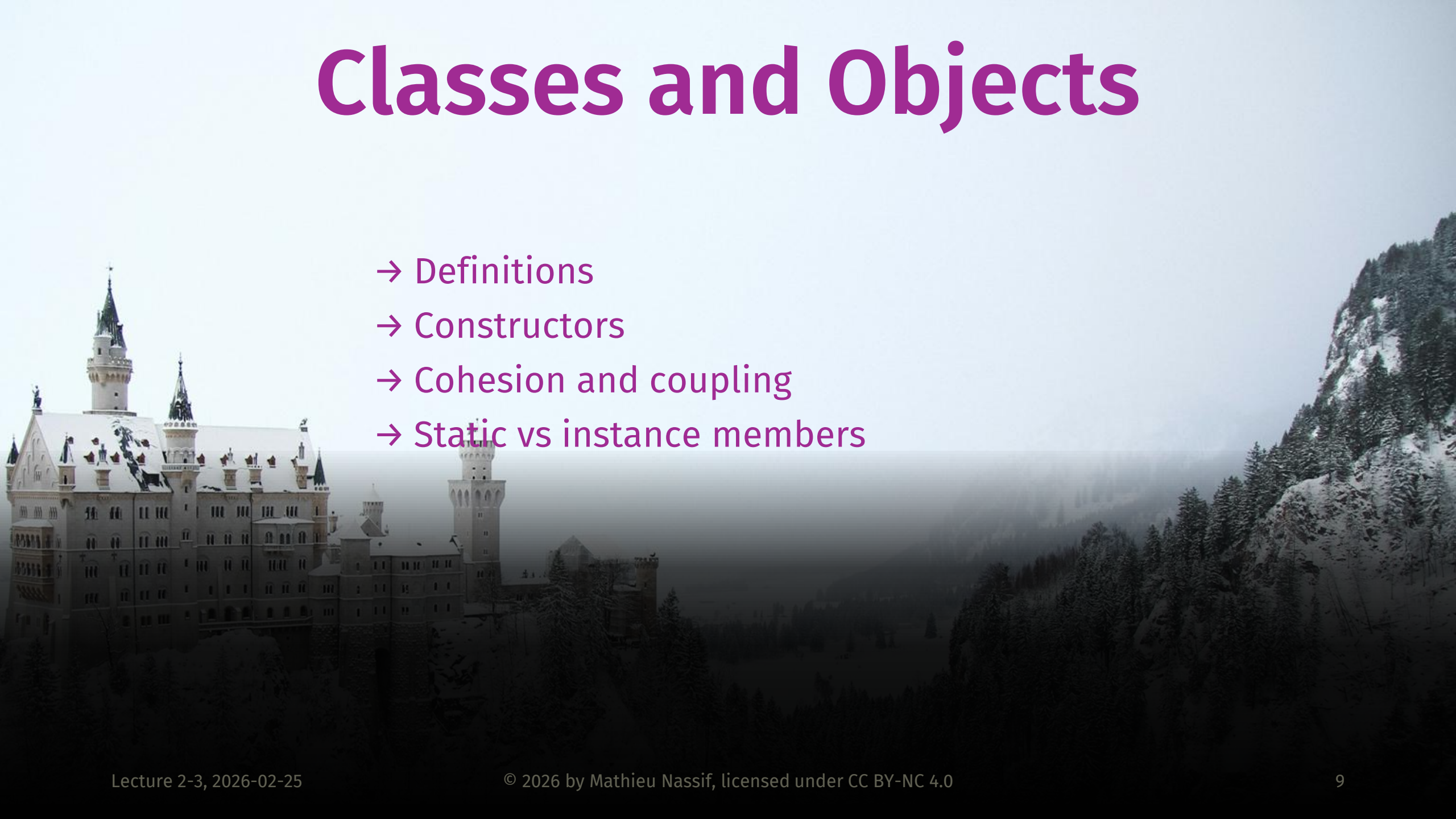
```
Integer.toString(12)
Boolean.toString(false)
```

- String to primitive type

```
Integer.parseInt("12")      // return int
Integer.valueOf("12")       // return Integer
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Classes and Objects



→ Definitions
→ Constructors
→ Cohesion and coupling
→ Static vs instance members

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Objects store data (attributes) and actions (methods).

A class is a blueprint to create similar objects.

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Classes capture abstractions

We want to build a course notebook.
It will contain notes taken during lectures.



Notebook

Reference to slides

Graphical interface

Author

Note

Summarizer

Lecture

Date

Student/User

Drawings

Course

Topic

Sharing platform

Storage

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Let’s start with the Notebook

```java
class NotebookV1 {

    String[] notes = new String[100]; // arrays cannot be resized in Java
    int numberOfNotes = 0;

    void writeNote(String note) {
        notes[numberOfNotes] = note;
        numberOfNotes++;
    }

    String readSingleNote(int noteNumber) {
        return notes[noteNumber];
    }

    String readAllNotes() {
        return String.join("\n\n", Arrays.copyOfRange(notes, 0, numberOfNotes));
    }
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Notes are object too

```java
public class Note {

    String content;

    public void setContent(String newContent) {
        content = newContent;
    }

    public String getContent() {
        return content;
    }
}
```



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Create objects with new

```
NotebookV2 notebook = new NotebookV2();
Note note1 = new Note();
note1.setContent("First note");
Note note2 = new Note();
note2.setContent("Second note");
```

Similar to a method call, but with the keyword new

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

The content of notes shouldn’t change

```
    final String content;

//  public void setContent(String newContent) {
//      content = newContent;
//  }
```

…but how to initialize it?

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Class constructors

A **constructor** is a special method used to initialize an object.




```
[modifiers*] ClassName ( [parameters*] ) [throws clauses*]
{
    [body]
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Class constructors

`new Note()` does 2 things:

1. creates a new `Note` object, then
2. calls the `Note` constructor with no argument.

Constructors don't return a value.

Classes have an implicit constructor if and only if there is no explicit constructor.

- no argument
- empty body

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Note’s constructor

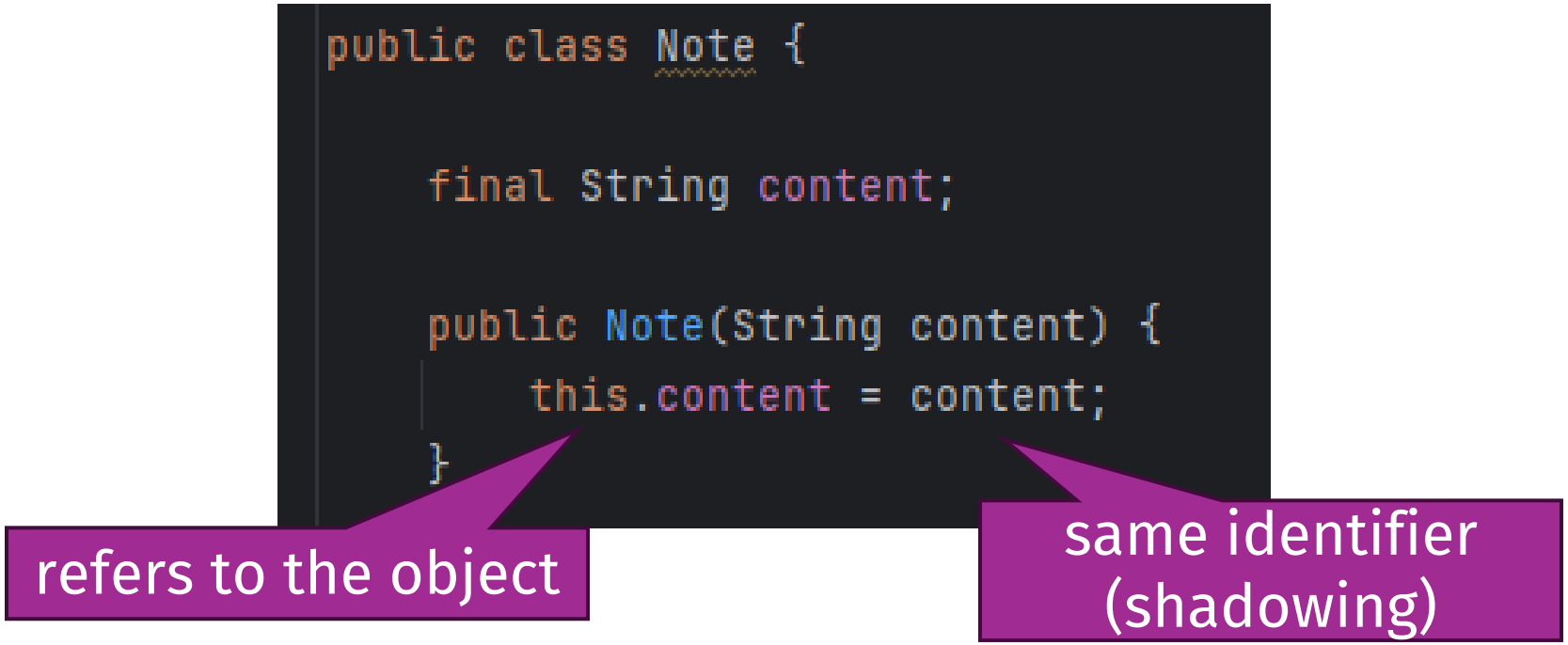
```java
public class Note {

    final String content;

    public Note(String content) {
        this.content = content;
    }
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

`this` keyword



- Not necessary to access members
- Except when a parameter is shadowing a field

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Overloading constructors

```java
public Note(String content) {
    this.content = content;
}

public Note(Note original, String newContent) {
    this.content = original.getContent() + newContent;
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

What about “destructors”?

- Deallocates memory when the object is no longer useful

- No destructor in Java
  - Garbage collector (GC) automatically clears memory
  - `finalize()` method, but it should not be used (deprecated)
  - more advanced options (`Cleaner`), but much beyond this course

- But it’s not just memory
  - May need to free other resources (e.g., network connection)
  - `AutoCloseable` provide a mechanism (for later in the course)

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

How to break the notebook

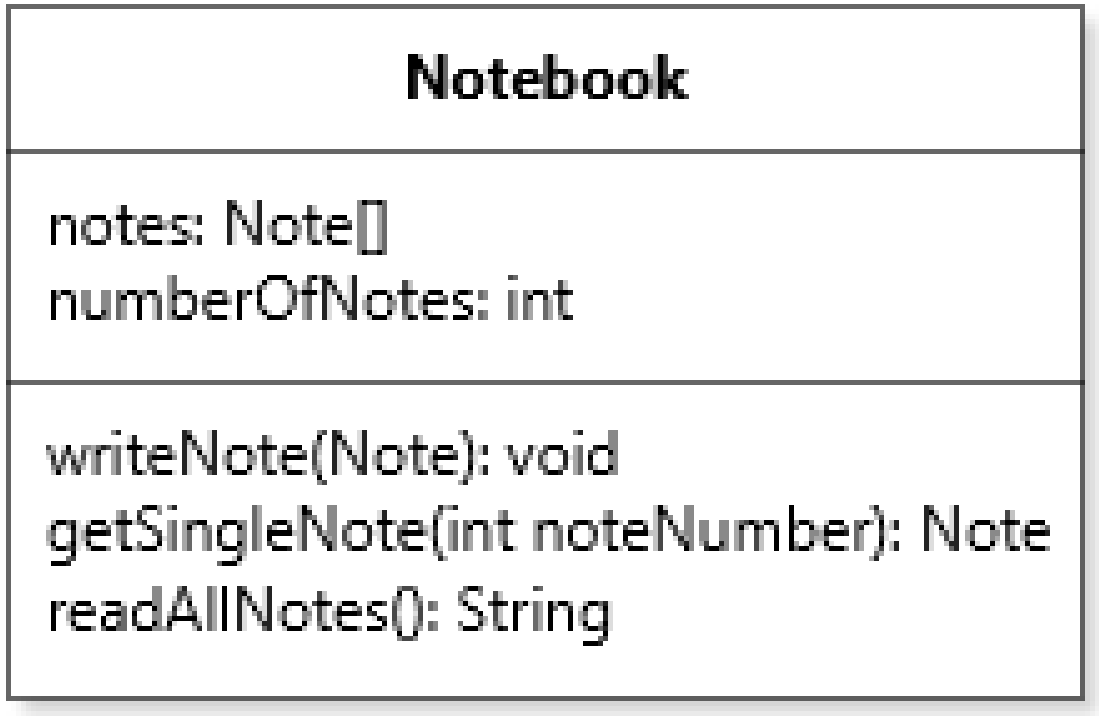
```
// messing up the notebook
notebook.numberOfNotes = 20;
notebook.notes = new Note[10];
notebook.notes[1] = new Note("My note");
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

How to break the notebook

Other classes should not modify fields directly

They should use methods instead



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Access modifiers control what information is visible outside the class

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Access modifiers

- `public`: visible to everyone (UML: +)
- `private`: invisible outside class (UML: -)
- `protected`: visible within package and to subclasses (UML: #)
- no modifier (package-private): visible within package (UML: ~)

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Access modifiers

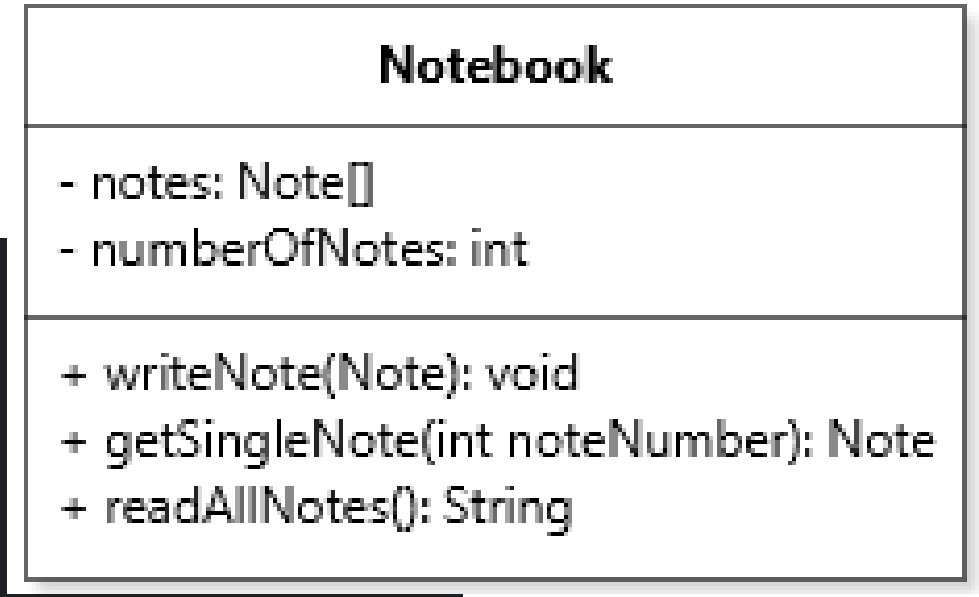
```java
public class NotebookV3 {

    private final Note[] notes = new Note[100];
    private int numberOfNotes = 0;

    public void writeNote(Note note) {...}

    public Note getSingleNote(int noteNumber) { return notes[noteNumber]; }

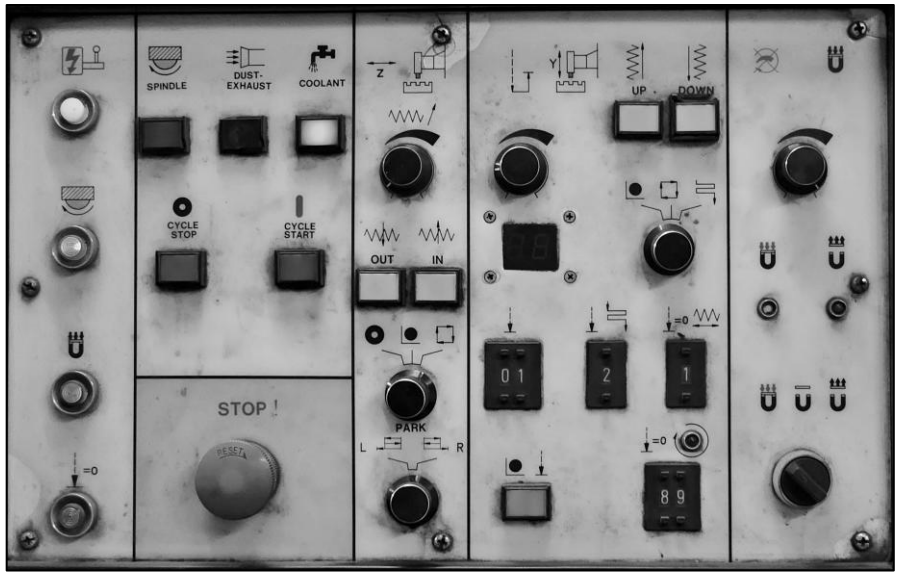
    public String readAllNotes() {...}
}
```

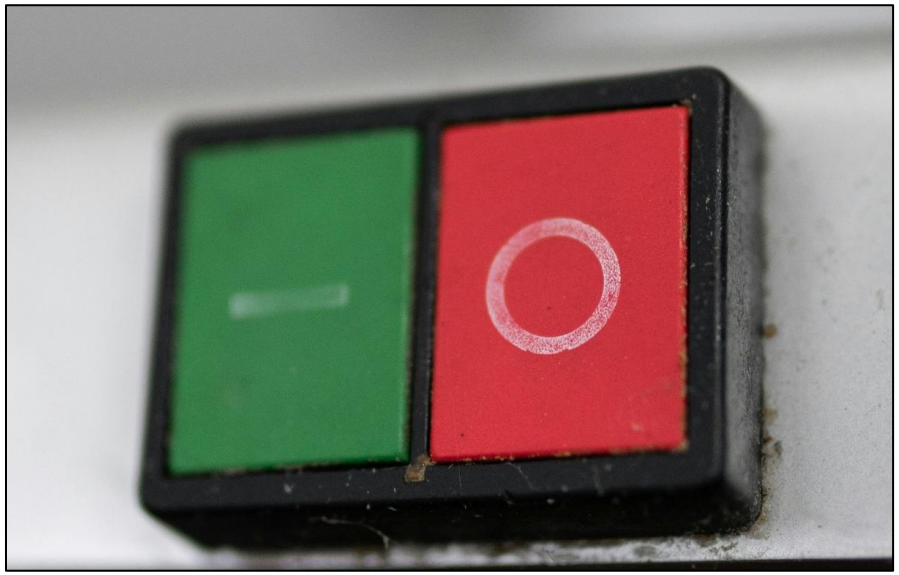


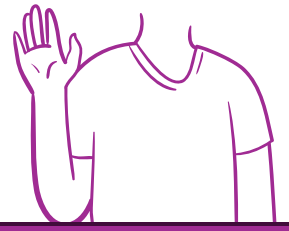
© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Access modifiers

How to choose which access modifier for each member?







Which machine is easier to operate?

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Access modifiers

How to choose which access modifier for each member?

- Make everything private, unless there's a **good reason** not to
- Make **fields private**
  - use **getters** (accessors) and **setters** (mutators) if necessary

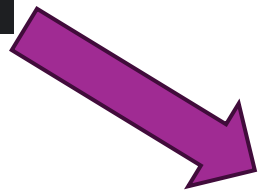
✓ Simplify "interface": list of members to interact with

✓ Allow to change implementation without affecting clients

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Access modifiers

```
Note[] notes = new Note[100];
int numberOfNotes = 0;
```



```
private final List<Note> notes = new ArrayList<>();
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Access modifiers

```java
private int slideNumber = -1;

public void setSlide(int slideNumber) {
    this.slideNumber = slideNumber;
}

public int getSlide() {
    return slideNumber;
}
```



Why use a getter and a setter here?

We may want to:

- check the value before assigning,
- update other fields on mutation,
- change the name of the field,
- …

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Design principles: Encapsulation and information hiding

Encapsulation

bundle data and actions in stable components

Information hiding

hide implementation details behind stable interfaces

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Design goal: low coupling, high cohesion

Coupling

Dependencies between components (classes)

Low coupling
Avoid complex dependency graphs

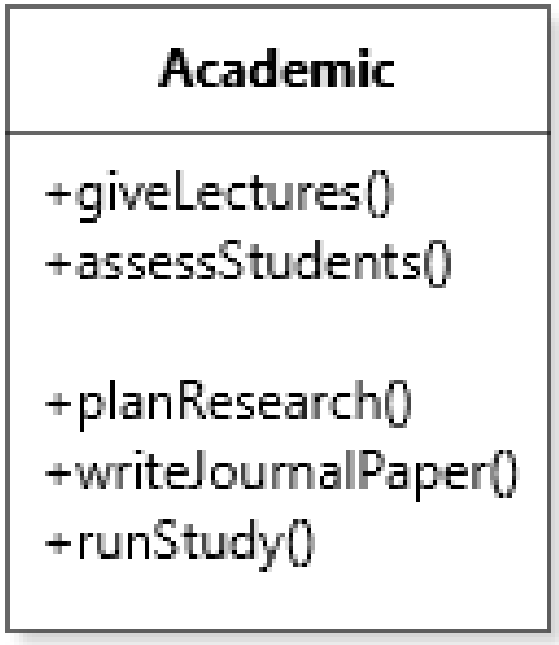
Cohesion

Relationships within components (classes)

High cohesion
Avoid unrelated elements in classes

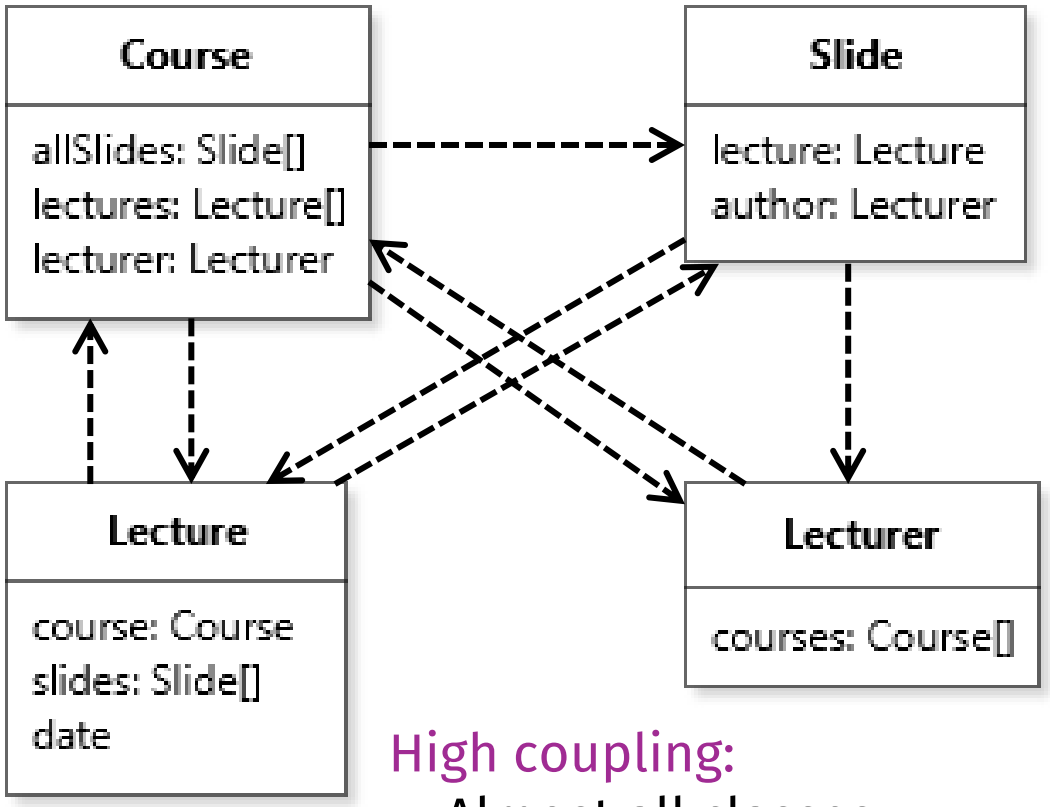
© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Design goal: low coupling, high cohesion



Low cohesion:

"teaching" and "researcher" concerns mixed together



High coupling:

Almost all classes depend on each other

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Static and instance members

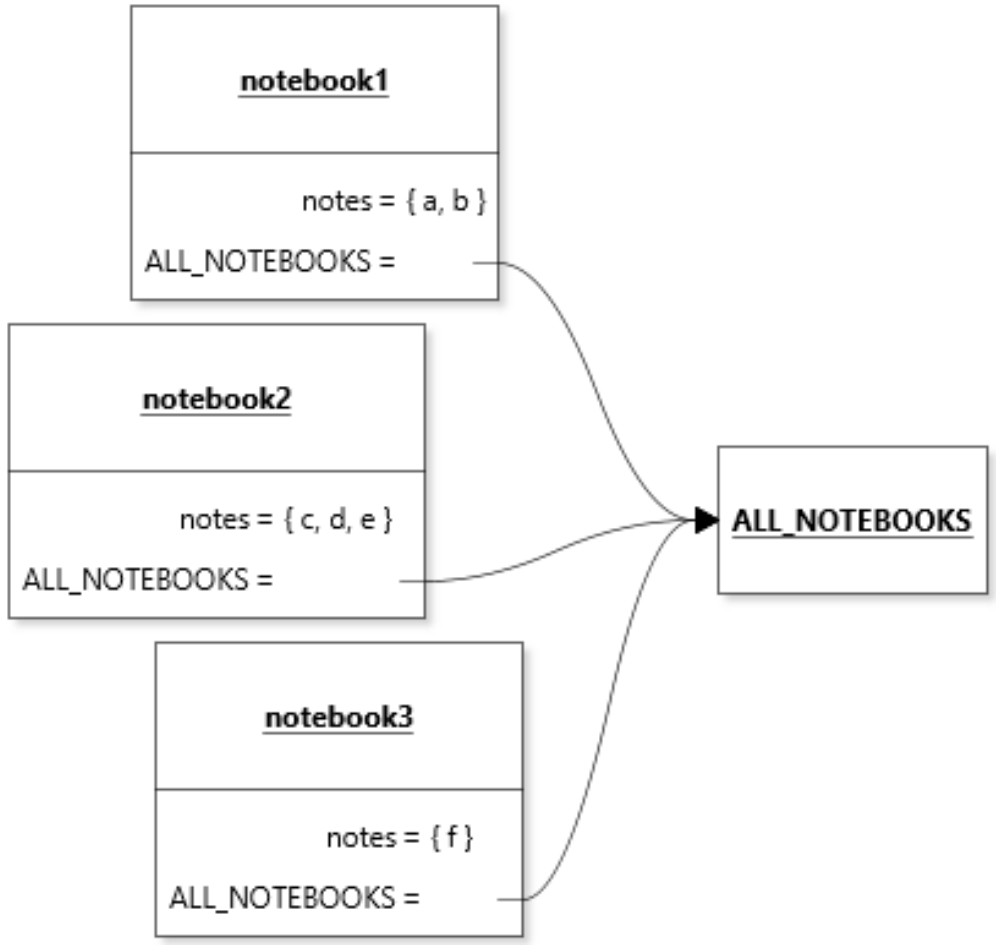
```java
private static final List<NotebookV4> ALL_NOTEBOOKS = new ArrayList<>();

public NotebookV4() {
    ALL_NOTEBOOKS.add(this);
}

public static int getCount() {
    return ALL_NOTEBOOKS.size();
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Static and instance members



Instance member

- each object has its own copy
- access from an object

Static member

- all objects have the same reference
- access from the class

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Static and instance members

```java
public static void addNoteToAllNotebooks(Note note) {
    notes.add(note);
}
```

can't access instance member in static method

```java
public static void addNoteToAllNotebooks(Note note) {
    for (NotebookV4 notebook : ALL_NOTEBOOKS) {
        notebook.notes.add(note);
    }
}
```

... unless on an explicit instance

```java
notebook.addNoteToAllNotebooks(new Note("common note"));
```

can access static member from instance

```java
NotebookV4.addNoteToAllNotebooks(new Note("common note"));
```

... but better to access from the class

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Multi-class Design



- → Packages
- → Dependencies in UML Class Diagrams

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Java Packages

Packages define a namespace in Java

Package names

- Components separated by '.' (e.g., `seng201.lecture2_3`)
- Fully qualified name (FQN) of a class is the package + class name
- Follows filesystem folder structure

class `lecture2_3.notebook.Notebook`
is defined in
file lecture2_1/notebook/Notebook.java

Classes in other packages must use import or FQN

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Java Packages

```java
package lecture2_3.notebook;
```

```java
import java.util.ArrayList;
//import java.util.List;

public class NotebookV4 {

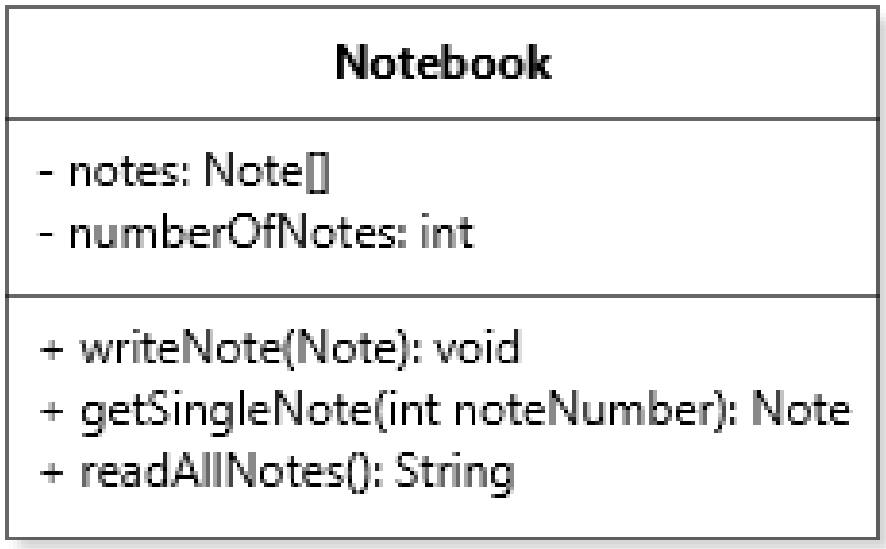
    private static final java.util.List<NotebookV4> ALL_NOTEBOOKS = new ArrayList<>();
```

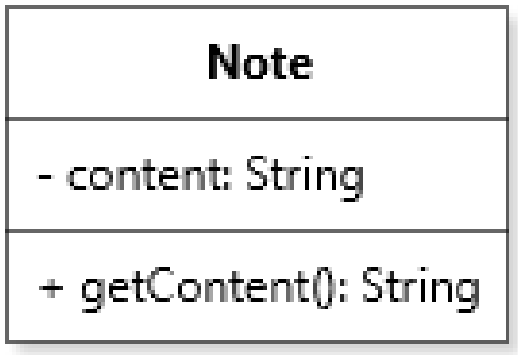
Fully qualified name (FQN)

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

UML Class Diagrams

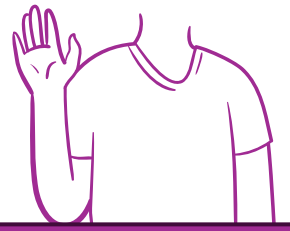
How can we indicate that the two classes depend on each other?





© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

UML Class Diagrams



What relationships exist between these concepts?

Vehicle

Driver

Car

Wheel

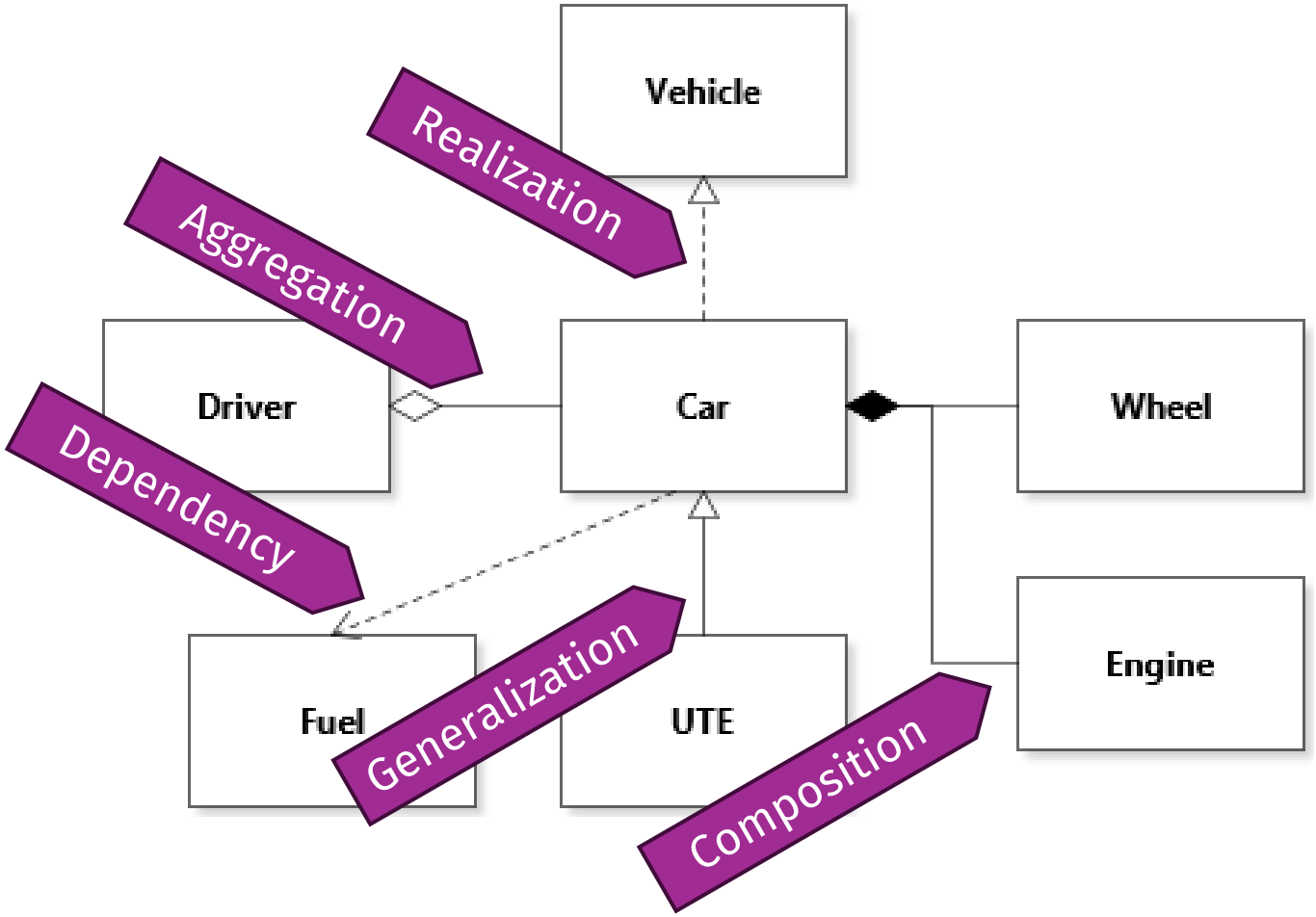
Fuel

UTE

Engine

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

UML Class Diagrams



Dependency:
Car “uses” Fuel

Aggregation:
Driver “contains” a Car

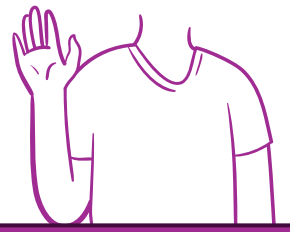
Composition:
Car “is made of” Wheels and Engine

Realization:
Car “fulfils” the spec of Vehicle (interface)

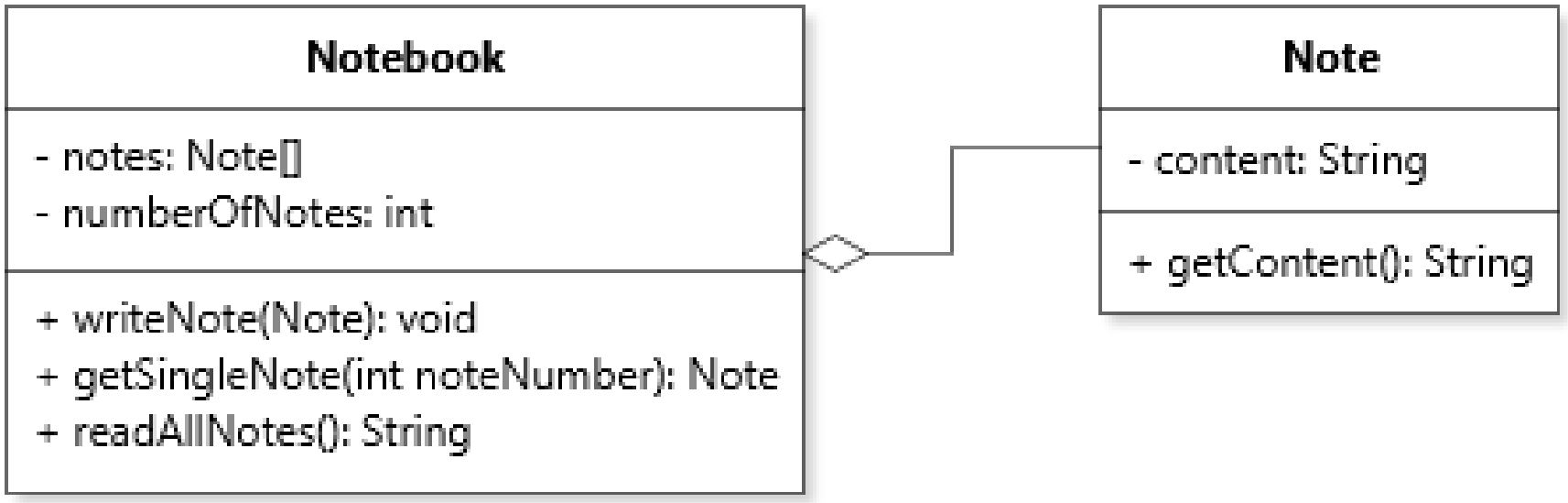
Generalization:
UTE “extends” Car (inheritance)

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

UML Class Diagrams



What relationships exist between these concepts?



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

RECAP

→ Strings

→ Classes: declarations, usage, design principles

→ UML Class diagrams



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0